Type	Input	Output	Definitions
Retrieve	x / x, y	{yes, no, continue}	Decides when to retrieve with $\mathcal{R}$
IsRel	x, d	{**relevant**, irrelevant}	d provides useful information to solve x.
IsSup	x, d, y	{**fully supported**, partially supported, no support}	All of the verification-worthy statement in y is supported by d.
IsUse	x, y	{**5**, 4, 3, 2, 1}	y is a useful response to x.

Table 1: Four types of reflection tokens used in SELF-RAG. Each type uses several tokens to represent its output values. The bottom three rows are three types of Critique tokens, and **the bold text** indicates the most desirable critique tokens. x, y, d indicate input, output, and a relevant passage, respectively.

Algorithm 1 SELF-RAG Inference

Require: Generator LM $\mathcal{M}$, Retriever $\mathcal{R}$, Large-scale passage collections $\{d_1, \ldots, d_N\}$
Input: input prompt x and preceding generation $y_{<t}$, **Output:** next output segment y_t
$\mathcal{M}$ predicts Retrieve given $(x, y_{<t})$
if Retrieve == Yes **then**
  Retrieve relevant text passages $\mathbf{D}$ using $\mathcal{R}$ given (x, y_{t-1}) ▷ Retrieve
  $\mathcal{M}$ predicts IsRel given x, d and y_t given $x, d, y_{<t}$ for each $d \in \mathbf{D}$ ▷ Generate
  $\mathcal{M}$ predicts IsSup and IsUse given x, y_t, d for each $d \in \mathbf{D}$ ▷ Critique
  Rank y_t based on IsRel , IsSup , IsUse ▷ Detailed in Section 3.3
else if Retrieve == No **then**
  $\mathcal{M}_{gen}$ predicts y_t given x ▷ Generate
  $\mathcal{M}_{gen}$ predicts IsUse given x, y_t ▷ Critique

Inference overview. Figure 1 and Algorithm 1 present an overview of SELF-RAG at inference. For every x and preceding generation $y_{<t}$, the model decodes a retrieval token to evaluate the utility of retrieval. If retrieval is not required, the model predicts the next output segment, as it does in a standard LM. If retrieval is needed, the model generates: a critique token to evaluate the retrieved passage's relevance, the next response segment, and a critique token to evaluate if the information in the response segment is supported by the passage. Finally, a new critique token evaluates the overall utility of the response.[4] To generate each segment, SELF-RAG processes multiple passages in parallel and uses its own generated reflection tokens to enforce soft constraints (Section 3.3) or hard control (Algorithm 1) over the generated task output. For instance, in Figure 1 (right), the retrieved passages d_1 is selected at the first time step since d_2 does not provide direct evidence (IsRel is Irrelevant) and d_3 output is only partially supported while d_1 are fully supported.

Training overview. SELF-RAG enables an arbitrary LM to generate text with reflection tokens by unifying them as next token predictions from the expanded model vocabulary (i.e., the original vocabulary plus reflection tokens). Specifically, we train the generator model $\mathcal{M}$ on a curated corpus with interleaving passages retrieved by a *retriever* $\mathcal{R}$ and reflection tokens predicted by a *critic* model $\mathcal{C}$ (summarized in Appendix Algorithm 2). We train $\mathcal{C}$ to generate reflection tokens for evaluating retrieved passages and the quality of a given task output (Section 3.2.1). Using the critic model, we update the training corpus by inserting reflection tokens into task outputs offline. Subsequently, we train the final generator model ($\mathcal{M}$) using the conventional LM objective (Section 3.2.2) to enable $\mathcal{M}$ to generate reflection tokens by itself without relying on the critic at inference time.

3.2 SELF-RAG TRAINING

Here, we describe the supervised data collection and training of two models, the critic $\mathcal{C}$ (Section 3.2.1) and the generator $\mathcal{M}$ (Section 3.2.2).

3.2.1 TRAINING THE CRITIC MODEL

Data collection for critic model. Manual annotation of reflection tokens for each segment is expensive (Wu et al., 2023). A state-of-the-art LLM like GPT-4 (OpenAI, 2023) can be effectively

[4]We follow Liu et al. (2023a) in using a "perceived" utility value that is independent of retrieved passages.

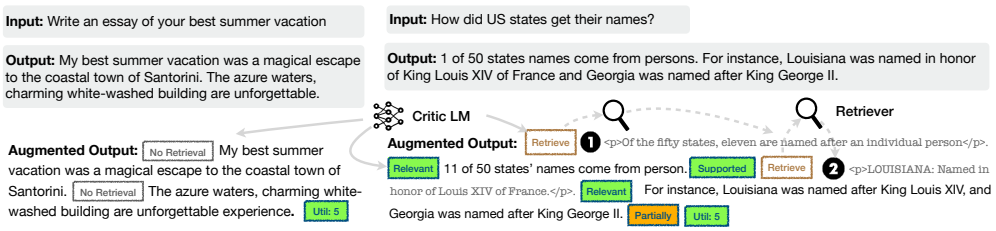

Figure 2: SELF-RAG training examples. The left example does not require retrieval while the right one requires retrieval; thus, passages are inserted. More examples are in Appendix Table 4.

used to generate such feedback (Liu et al., 2023b). However, depending on such proprietary LMs can raise API costs and diminish reproducibility (Chen et al., 2023). We create supervised data by prompting GPT-4 to generate reflection tokens and then distill their knowledge into an in-house $\mathcal{C}$. For each group of reflection tokens, we randomly sample instances from the original training data: $\{X^{sample}, Y^{sample}\} \sim \{X, Y\}$. As different reflection token groups have their own definitions and input, as shown in Table 1, we use different instruction prompts for them. Here, we use Retrieve as an example. We prompt GPT-4 with a type-specific instruction ("Given an instruction, make a judgment on whether finding some external documents from the web helps to generate a better response.") followed by few-shot demonstrations I the original task input x and output y to predict an appropriate reflection token as text: $p(r|I, x, y)$. Manual assessment reveals that GPT-4 reflection token predictions show high agreement with human evaluations. We collect 4k-20k supervised training data for each type and combine them to form training data for $\mathcal{C}$. Appendix Section D shows the full list of instructions, and A.1 contains more details and our analysis.

Critic learning. After we collect training data $\mathcal{D}_{critic}$, we initialize $\mathcal{C}$ with a pre-trained LM and train it on $\mathcal{D}_{critic}$ using a standard conditional language modeling objective, maximizing likelihood:

$$\max_{\mathcal{C}} \mathbb{E}_{((x,y),r)\sim\mathcal{D}_{critic}} \log p_{\mathcal{C}}(r|x, y), \ r \text{ for reflection tokens.} \tag{1}$$

Though the initial model can be any pre-trained LM, we use the same one as the generator LM (i.e., Llama 2-7B; Touvron et al. 2023) for $\mathcal{C}$ initialization. The critic achieves a higher than 90% agreement with GPT-4-based predictions on most reflection token categories (Appendix Table 5).

3.2.2 TRAINING THE GENERATOR MODEL

Data collection for generator. Given an input-output pair (x, y), we augment the original output y using the retrieval and critic models to create supervised data that precisely mimics the SELF-RAG inference-time process (Section 3.1). For each segment $y_t \in y$, we run $\mathcal{C}$ to assess whether additional passages could help to enhance generation. If retrieval is required, the retrieval special token Retrieve =Yes is added, and $\mathcal{R}$ retrieves the top K passages, **D**. For each passage, $\mathcal{C}$ further evaluates whether the passage is relevant and predicts IsRel. If a passage is relevant, $\mathcal{C}$ further evaluates whether the passage supports the model generation and predicts IsSup. Critique tokens IsRel and IsSup are appended after the retrieved passage or generations. At the end of the output, y (or y_T), $\mathcal{C}$ predicts the overall utility token IsUse, and an augmented output with reflection tokens and the original input pair is added to $\mathcal{D}_{gen}$. See the example training data in Figure 2.

Generator learning. We train the generator model $\mathcal{M}$ by training on the curated corpus augmented with reflection tokens $\mathcal{D}_{gen}$ using the standard next token objective:

$$\max_{\mathcal{M}} \mathbb{E}_{(x,y,r)\sim\mathcal{D}_{gen}} \log p_{\mathcal{M}}(y, r|x). \tag{2}$$

Unlike $\mathcal{C}$ training (Eq. 1), $\mathcal{M}$ learns to predict the target output as well as the reflection tokens. During training, we mask out the retrieved text chunks (surrounded by and in Figure 2) for loss calculation and expand the original vocabulary $\mathcal{V}$ with a set of reflection tokens {Critique, Retrieve}.

Connections to prior work on learning with critique. Recent work incorporates additional critique (feedback) during training, e.g., RLHF (Ouyang et al. 2022) via PPO. While PPO relies on

separate reward models during training, we compute critique offline and directly insert them into the training corpus, where the generator LM is trained with a standard LM objective. This significantly reduces training costs compared to PPO. Our work also relates to prior work that incorporates special tokens to control generation (Keskar et al., 2019; Lu et al., 2022; Korbak et al., 2023). Our SELF-RAG learns to generate special tokens *to evaluate its own prediction* after each generated segment, enabling the use of a soft re-ranking mechanism or hard constraints at inference (discussed next).

3.3 SELF-RAG INFERENCE

Generating reflection tokens to self-evaluate its own output makes SELF-RAG controllable during the inference phase, enabling it to tailor its behavior to diverse task requirements. For tasks demanding factual accuracy (Min et al., 2023), we aim for the model to retrieve passages more frequently to ensure that the output aligns closely with the available evidence. Conversely, in more open-ended tasks, like composing a personal experience essay, the emphasis shifts towards retrieving less and prioritizing the overall creativity or utility score. In this section, we describe approaches to enforce control to meet these distinct objectives during the inference process.

Adaptive retrieval with threshold. SELF-RAG dynamically decides when to retrieve text passages by predicting [Retrieve]. Alternatively, our framework allows a threshold to be set. Specifically, if the probability of generating the [Retrieve]=`Yes` token normalized over all output tokens in [Retrieve] surpasses a designated threshold, we trigger retrieval (details in Appendix Section A.3).

Tree-decoding with critique tokens. At each segment step t, when retrieval is required, based either on hard or soft conditions, $\mathcal{R}$ retrieves K passages, and the generator $\mathcal{M}$ processes each passage in parallel and outputs K different continuation candidates. We conduct a segment-level beam search (with the beam size=B) to obtain the top-B segment continuations at each timestamp t, and return the best sequence at the end of generation. The score of each segment y_t with respect to passage d is updated with a critic score $\mathcal{S}$ that is the linear weighted sum of the normalized probability of each [Critique] token type. For each critique token group G (e.g., [IsRel]), we denote its score at timestamp t as s_t^G, and we compute a segment score as follows:

$$f(y_t, d, [\text{Critique}]) = p(y_t|x, d, y_{<t})) + \mathcal{S}([\text{Critique}]), \text{where} \tag{3}$$

$$\mathcal{S}([\text{Critique}]) = \sum_{G \in \mathcal{G}} w^G s_t^G \text{ for } \mathcal{G} = \{[\text{IsRel}], [\text{IsSup}], [\text{IsUse}]\}, \tag{4}$$

where $s_t^G = \frac{p_t(\hat{r})}{\sum_{i=1}^{N^G} p_t(r_i)}$ stands for the generation probability of the most desirable reflection token $\hat{r}$ (e.g., [IsRel] =`Relevant`) for the critique token type G with N^G distinct tokens (that represent different possible values for G). The weights w^G in Eq. 4 are hyperparameters that can be adjusted at inference time to enable customized behaviors at test time. For instance, to ensure that result y is mostly supported by evidence, we can set a weight term for the [IsSup] score higher, while relatively lowering weights for other aspects. Alternatively, we could further enforce hard constraints during decoding using [Critique]. Instead of using a soft reward function in Eq. 4, we could explicitly filter out a segment continuation when the model generates an undesirable [Critique] token (e.g., [IsSup] =`No support`) . Balancing the trade-off between multiple preferences has been studied in RLHF (Touvron et al., 2023; Wu et al., 2023), which often requires training to change models' behaviors. SELF-RAG tailors an LM with no additional training.

4 EXPERIMENTS

4.1 TASKS AND DATASETS

We conduct evaluations of our SELF-RAG and diverse baselines on a range of downstream tasks, holistically evaluating outputs with metrics designed to assess overall correctness, factuality, and fluency. Throughout these experiments, we conduct zero-shot evaluations, where we provide instructions describing tasks without few-shot demonstrations (Wei et al., 2022; Sanh et al., 2022). Details of our experiments' settings, including test-time instructions, are available in the Appendix Section B.1.

Closed-set tasks include two datasets, i.e., a fact *verification dataset* about public health (**PubHealth**; Zhang et al. 2023) and a *multiple-choice reasoning dataset* created from scientific exams (**ARC-**